```
class Demo
  @@var = 99
  CONST = 1.23

  private def private_method
  end

  protected def protected_method
  end

  public def public_method
    @inst = 1
    i = 1
    j = 2
    local_variables
  end

  def Demo.class_method
  end
end

Demo.private_instance_methods(false)   # => [:private_method]
Demo.protected_instance_methods(false) # => [:protected_method]
Demo.public_instance_methods(false)    # => [:public_method]
Demo.singleton_methods(false)          # => [:class_method]
Demo.class_variables                   # => [:@@var]
Demo.constants(false)                  # => [:CONST]

demo = Demo.new
demo.instance_variables                # => []
# Get 'public_method' to return its local variables
# and set an instance variable
demo.public_method                     # => [:i, :j]
demo.instance_variables                # => [:@inst]
```

You may be wondering what all the false parameters were in the previous code. These reflection methods will by default recurse into parent classes, their parents, and so on, up the ancestor chain. Passing in false stops this kind of prying.

Given a list of method names, we may now be tempted to try calling them. Fortunately, that's easy with Ruby.

Calling Methods Dynamically

The send method lets you tell any object to invoke a method by name. The first argument is a symbol or a string representing the name, and any remaining arguments are passed along to the method of that name. For example:

```
"John Coltrane".send(:length)          # => 13
"Miles Davis".send("sub", /iles/, '.') # => "M. Davis"
```

There are two twists to send. For example, your class might define its own send method if it wanted to send something somewhere. Ruby provides the __send__ method, defined in BasicObject, which is identical to send and meant to be used in cases where send might have been overwritten.

The send method does not enforce method access, meaning that it will happily send to a method that is private or protected. You might reasonably think that undermines the point of